

LLM-Powered SOC Assistant

A Natural Language Interface for Open-Source Security
Monitoring

Progress Report 5

Umesh Kalupathirannehelage – 300389749

Applied Research Project

CSIS4495-071

1. Work Log Table

Student Name: Umesh Kalupathirannehelage

Date	Hours	Description of Work Done
Nov 10 2025	2	Extended the schema hint and backend logic to introduce severity buckets (low, medium, high, critical) based on rule_level, so LLM-generated SQL can correctly filter and group by severity.
Nov 12 2025	2	Implemented a new /stats/severity API endpoint that aggregates events by severity for the last x hours and returns labels, counts, and percentages for dashboards.
Nov 13 2025	2	Integrated the "Severity distribution (last 24h)" Chart.js visualization in the UI, wired it to /stats/severity, and verified that the pie chart reflects recent alert volumes correctly.
Nov 15 2025	2	Added /filters/agents backend endpoint to return distinct agent_name values and hooked it to a dynamic Agent dropdown in the frontend.
Nov 16 2025	2.5	Connected Agent and Severity dropdown selections to query generation so natural-language questions are automatically constrained to the chosen agent and severity range.
Nov 18 2025	2	Refactored main.py for better readability by grouping config, helpers, and API endpoints; strengthened the schema hint with explicit parentheses for failed/invalid/denied conditions to avoid boolean logic bugs.
Nov 21 2025	2	Performed end-to-end testing of the new filters and severity dashboards; confirmed SQL output and result rows match the selected filters and captured screenshots for the final demo and report.

2. Description of Work Done

Oct 23- Nov 8, 2025

During this reporting period, my focus shifted toward making the SOC Assistant more analyst-friendly through interactive filtering, improved SQL accuracy, enhanced charting, and a more scalable UI structure. The primary goal was to evolve the dashboard from a static query tool into an adaptive, analyst-driven interface that responds to environment data and security context in real time.

The first major enhancement was the introduction of **dynamic agent filtering**. I implemented a new `/filters/agents` backend endpoint that fetches distinct agent names directly from the PostgreSQL database. This enabled the frontend to populate the “Agent” dropdown dynamically, ensuring the UI always reflects available Wazuh agents without hard coding. To support this, I updated the `SCHEMA_HINT` with stricter boolean-logic rules, ensuring the LLM always applies grouping (parentheses) when combining AND + OR conditions—preventing incorrect query expansion and fixing the issue where unrelated agents appeared in filtered results.

Next, I introduced **severity-based filtering and visual drilldowns**, allowing users to refine natural-language queries by selecting “Low,” “Medium,” “High,” or “Critical” severity levels. On the visualization side, I added a dedicated endpoint (`/stats/severity`) that returns severity counts for the last 24 hours, enabling real-time pie-chart rendering. I also added client-side logic to support drill-down actions, where chart segments can drive the next query, making the dashboard more interactive and SOC-oriented.

To simplify the UI for better usability, I removed the time-range dropdown, reducing unnecessary complexity while keeping agent and severity filters. I ensured that these filters now persist and intelligently modify the natural-language prompt before sending it to the LLM, resulting in consistent and accurate SQL generation.

I also enhanced backend validation to fix issues with IP-address handling and added SQL guards to prevent the LLM from injecting malformed conditions. Several UI improvements were made, including better error surfaces, updated layout spacing, and ensuring that ETL timestamp labels (“Last updated” / “Next update”) refresh correctly after both cron-based and manual ETL executions.

Together, these improvements significantly improved query accuracy, increased dashboard interactivity, and allowed the SOC Assistant to scale with new agents, severities, and data sources without code changes.

Under the implementations folder,

- **main.py (updated)** – Added /filters/agents endpoint, strengthened SCHEMA_HINT with explicit boolean-logic rules, fixed agent filtering issues, added severity-driven logic, improved SQL validation, and refined error handling.
- **ui/index.html (updated)** – Integrated dynamic agent dropdown, simplified layout by removing time-range selector, added severity filter select, updated interactive buttons, and improved ETL status display.
- **ui/script/app.js (updated)** – Implemented dynamic agent loading, applied filter-aware NL query injection, improved chart rendering, added severity-drill-down support, and fixed row rendering edge cases.

Under the documents folder

- **Project_UKa749_Report5.pdf** – This progress report document.